Smart Home

Artículo técnico del proyecto

Sistema inteligente de monitoreo y control energético residencial

Karen Daniela Holguín Cruz

Natalia Chala Chala

Kevin Stiven López Amaya

SENA

Servicio Nacional de Aprendizaje – SENA
Análisis y Desarrollo de Software (ADSO) · Ficha 3145555
Instructor: José de Jesús Motta Vargas
Octubre de 2026

Índice

Este artículo presenta el problema, la metodología, la solución técnica, las pruebas ejecutadas y los resultados del proyecto Smart Home, un sistema de monitoreo y control energético residencial basado en IoT.

Contenido	Página
Título, autores, resumen y palabras clave	3
1. Introducción y problema abordado	4
2. Metodología de desarrollo	5
3. Desarrollo de la solución	6
4. Pruebas y resultados de ejecución	7
5. Resultados y discusión	8
6. Conclusiones	8
Referencias bibliográficas	9

Smart Home
Sistema inteligente de monitoreo y control energético residencial

Karen Daniela Holguín Cruz · Natalia Chala Chala · Kevin Stiven López Amaya

SENA · Análisis y Desarrollo de Software (ADSO) · Ficha 3145555
Instructor: José de Jesús Motta Vargas
Artículo técnico del proyecto · Octubre de 2026

Resumen

El proyecto Smart Home aborda la problemática del sobreconsumo y la falta de visibilidad del gasto energético residencial mediante una solución integral distribuida que combina Internet de las Cosas (IoT) y desarrollo de software. El sistema integra un backend desarrollado en Node.js (NestJS/Express), una base de datos relacional PostgreSQL con Liquibase y aislamiento mediante Row Level Security (RLS), un broker de mensajería MQTT para la comunicación con dispositivos inteligentes (ESP32 y Shelly 1PM Gen4) y una pasarela de WebSockets para la actualización en tiempo real. La arquitectura permite la administración de hogares, la asignación de roles (OWNER, MEMBER, GUEST), el control conmutado de cargas y el análisis de telemetría eléctrica en kilovatios-hora (kWh) y costo estimado en pesos (COP). Los resultados presentados corresponden al diseño técnico, las pruebas de integración API/IoT documentadas y la corrección de flujos de control y seguridad. Se concluye que la plataforma ofrece un núcleo robusto y funcional listo para su despliegue y validación en entornos reales.

Palabras clave

Smart Home; IoT; MQTT; WebSockets; NestJS; PostgreSQL; Row Level Security; telemetría eléctrica; monitoreo energético.

1. Introducción y problema abordado

El consumo de energía eléctrica en el sector residencial representa una porción significativa del gasto de los hogares colombianos. Sin embargo, los usuarios carecen de herramientas en tiempo real para supervisar la demanda de sus electrodomésticos, dependiendo de la lectura mensual consolidada enviada por la empresa prestadora del servicio. Esta ausencia de información inmediata limita la capacidad de identificar equipos de alto consumo, prever cobros elevados o detectar anomalías eléctricas.

El proyecto Smart Home se desarrolla para integrar la medición de telemetría eléctrica, el control remoto de dispositivos y la gestión familiar de hogares dentro de una solución accesible desde plataformas web y aplicaciones móviles.

Problema y objetivo

El problema abordado es la falta de control proactivo y visibilidad sobre el consumo eléctrico residencial en tiempo real, junto con la necesidad de garantizar la privacidad y el aislamiento de datos entre distintos hogares.

El objetivo general es desarrollar un sistema de monitoreo y control energético residencial con arquitectura desacoplada, comunicación IoT en tiempo real y seguridad a nivel de datos. Sus objetivos específicos comprenden:

- Organizar el modelo relacional de datos de usuarios, hogares, zonas y dispositivos.
- Implementar autenticación segura mediante JWT con revocación de sesiones.
- Integrar la ingesta de datos IoT vía MQTT y la distribución en tiempo real mediante WebSockets.
- Proveer un panel de control e interfaz gráfica para la supervisión y generación de alertas por sobreconsumo.
- Garantizar el aislamiento de datos multi-inquilino (multi-tenant) mediante políticas RLS en PostgreSQL.

Usuarios y alcance

Rol	Participación en el sistema
Propietario (OWNER)	Administra el hogar, configura tarifas/estratos, vincula/elimina dispositivos, gestiona invitaciones y miembros, y controla dispositivos.
Miembro (MEMBER)	Visualiza el consumo del hogar, consulta reportes, recibe alertas y opera el control de encendido/apagado (ON/OFF) de los dispositivos.
Invitado (GUEST)	Acceso de solo lectura para consultar el estado del hogar y los indicadores de consumo energético.

El alcance incluye dispositivos inteligentes (ESP32 / Shelly 1PM Gen4) y estimación para dispositivos manuales; la gestión de otros servicios como agua o gas queda fuera del alcance actual.

2. Metodología de desarrollo

Se describe un proceso de desarrollo iterativo orientado a funcionalidades, alineado con las fases del programa ADSO del SENA.

Análisis y modelado

A partir de la Especificación de Requisitos de Software (SRS), se definieron las historias de usuario, reglas de negocio y restricciones operativas. El modelado UML (casos de uso, clases, secuencia, componentes y despliegue) y el modelo Entidad-Relación orientaron la estructuración de la base de datos SmartHomeDB.

Implementación e integración

La construcción se distribuye de la siguiente manera:

- **Persistencia:** migraciones versionadas mediante Liquibase sobre PostgreSQL 14+, aplicando políticas RLS.
- **Backend:** Node.js con NestJS/Express que implementa la lógica de negocio, controladores REST y adaptadores MQTT/WebSockets.
- **Dispositivos IoT:** carga de firmware en microcontroladores ESP32 y configuración de dispositivos reales Shelly 1PM Gen4 sobre protocolo MQTT.
- **Frontend:** interfaz cliente desarrollada en React/TypeScript para el consumo de la API y canal de WebSockets.

ARQUITECTURA DE INTEGRACIÓN		
CAPA IoT	**CAPA BACKEND**	**CAPA CLIENTE**
• ESP32 / Shelly • Sensor SCT-013 • Relé 5V DC	• Node.js (NestJS) • Broker Mosquitto (MQTT) • PostgreSQL + RLS	• Dashboard Web • App Mobile • WebSockets

Criterios de evaluación

Criterio	Evidencia que permite evaluarlo
Integridad y negocio	El backend valida la existencia del usuario, restringe invitaciones duplicadas (HTTP 409) y mantiene la coherencia de estados.
Seguridad funcional	Aplicación de RBAC (Control de Acceso Basado en Roles) y políticas RLS que aíslan los datos por usuario/hogar en la BD.
Sincronización realtime	Ingesta de tramas MQTT de consumo e instantánea difusión vía pasarela WebSockets (RealtimeGateway).
Mantenibilidad	Código modular en capas, scripts Liquibase reproducibles y documentación de endpoints con Swagger.

3. Desarrollo de la solución

Backend y arquitectura en capas

El backend en Node.js (NestJS) opera bajo una arquitectura modular en capas. Los módulos agrupados incluyen **auth**, **homes**, **devices**, **consumption** y **notifications**. Dentro de cada módulo, los controladores reciben solicitudes HTTP, los servicios ejecutan la lógica de negocio y las reglas de validación, y los repositorios/ORM interactúan con PostgreSQL.

Se incorporaron patrones como Data Transfer Objects (DTOs) para la validación de entrada, interceptores para la gestión de excepciones y adaptadores para la suscripción MQTT (MqttDeviceStatusSubscriber) y emisión de tiempo real (RealtimeGateway).

Persistencia y seguridad (RLS)

PostgreSQL conserva las relaciones del sistema. Las migraciones de Liquibase garantizan un esquema reproducible. Para asegurar la privacidad de los datos entre diferentes usuarios y hogares, se implementó Row Level Security (RLS) en el motor relacional, separando los roles de conexión técnica [2]:

- **smarthome_app:** usuario runtime del backend.
- **smarthome_ingest:** usuario exclusivo para la ingesta masiva de telemetría IoT.

Protocolos de comunicación IoT

La comunicación con los dispositivos físicos (Shelly 1PM Gen4) y simulados se realiza mediante MQTT [1] sobre el broker Eclipse Mosquitto.

Tipo de tópico	Estructura
Telemetría	smarthome/devices/{deviceId}/telemetry
Comandos (TURN_ON / TURN_OFF)	smarthome/devices/{deviceId}/command

4. Pruebas y resultados de ejecución

El desarrollo técnico fue validado mediante un plan de ejecución de pruebas sobre la API REST y el pipeline IoT, utilizando Swagger UI y consolas de registro del servidor [5].

Resumen de pruebas ejecutadas

Caso de prueba	Componente evaluado	Estado
CP-01: Autenticación	POST /api/v1/auth/login	✔ **Aprobado**
CP-02: Consulta casas	GET /api/v1/homes	✔ **Aprobado**
CP-03: Conflict 409	POST /invitations	✔ **Aprobado**
CP-04: Invitación 201	POST /invitations	✔ **Aprobado**
CP-05: Miembros del hogar	GET /members	✔ **Aprobado**
CP-06: Ingesta MQTT	MqttSubscriber / WS	✔ **Aprobado**
CP-07: Telemetría	GET /devices	✔ **Aprobado**

Resultados destacados de las pruebas

- **Autenticación y seguridad (CP-01):** la llamada a POST /api/v1/auth/login validó correctamente las credenciales del usuario (nataliachala479@gmail.com), retornando el token JWT necesario para autorizar solicitudes subsiguientes.
- **Validación de reglas de negocio (CP-03 y CP-04):** al intentar invitar a un usuario previamente vinculado a un hogar (POST /invitations), el sistema respondió de forma adecuada con el código 409 Conflict y el mensaje: «El usuario ya tiene una invitación o membresía activa en este hogar». La invitación a un nuevo usuario (klddaniela8@gmail.com) retornó el código 201 Created con estado «PENDING».
- **Ingesta IoT y tiempo real (CP-06 y CP-07):** el componente MqttDeviceStatusSubscriber procesó correctamente la telemetría enviada por el dispositivo real Shelly 1PM Gen4 (shelly1pmg4-d885acf06990), registrando en los logs del servidor *Consumo guardado=true* y confirmando la presencia del cliente WebSockets en la pasarela RealtimeGateway. La consulta de dispositivos (GET /devices) retornó el estado operativo «ONLINE» con lectura de potencia activa de 101 W para la lámpara de sala.

5. Resultados y discusión

Resultados técnicos documentados

El sistema cuenta con un backend operativo al 90–95 %, soporte funcional local al 85 % y un frontend web integrado al 70 %. Se lograron corregir inconsistencias previas eliminando el atributo redundante *device_type*, implementando la revocación masiva de sesiones (*logout-all*) y aislando las cadenas de conexión IoT mediante la variable *DATABASE_IOT_URL*.

Discusión

La desacoplación del pipeline de mensajes MQTT frente al servidor API REST permite procesar volúmenes continuos de datos de consumo sin bloquear las peticiones de los usuarios. Sin embargo, se identifica que para el despliegue en entornos de producción cloud se debe avanzar en la configuración de protocolos seguros MQTTS (TLS/SSL) y en la implementación del flujo de emparejamiento móvil (Pairing) vía Bluetooth Low Energy (BLE) / Wi-Fi provisioning.

6. Conclusiones

Smart Home ofrece una arquitectura de software robusta, desacoplada y segura para la administración, monitoreo y control del consumo energético residencial.

La integración de Node.js, PostgreSQL con políticas RLS y protocolos MQTT/WebSockets demostró ser altamente eficiente para procesar telemetría eléctrica con baja latencia y con estricta privacidad de datos entre hogares.

Las pruebas funcionales realizadas sobre la API REST y la pasarela de dispositivos confirmaron el cumplimiento de las reglas de negocio, la correcta gestión de respuestas HTTP y la estabilidad en la comunicación con hardware real.

Referencias bibliográficas

[1] OASIS Standard. (2019). *MQTT Version 5.0*. OASIS Open. Disponible en: docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation: Row Level Security Policies*. Disponible en: postgresql.org/docs/current/ddl-rowsecurity.html

[3] NestJS. (2026). *A progressive Node.js framework for building efficient, reliable and scalable server-side applications*. Disponible en: docs.nestjs.com

[4] Equipo Smart Home. (2026). Especificación de Requisitos de Software (SRS) y Plan Maestro del Proyecto Smart Home. Documentos internos del proyecto, SENA Ficha 3145555.

[5] Equipo Smart Home. (2026). Plan e Informe de Ejecución de Pruebas del Backend y Pasarela IoT. Documentación técnica interna y evidencias de ejecución API/Swagger.